STL10 图像分类模型优化实验报告

人工智能原理课程 • 项目二 • STL10 图像分类

院　　系：	自动化系
班　　级：	自 52 班
学 生 姓 名：	刘江宇
学　　号：	2024013215
课 程 名 称：	人工智能原理

目录

1 实验目标 **2**

2 实验设置 **2**

3 实验分组与网络结构说明 **2**

4 实验结果概览 **3**

5 训练过程与结果分析 **4**

5.1 Baseline：测试集分类报告与混淆矩阵 4

5.2 （一）baseline → exp1：数据增强 5

5.3 （二）exp1 → exp2：BatchNorm 5

5.4 （三）exp2 → exp3：Dropout（与 BN 路径对照） 6

5.5 （四）exp2 → exp4：优化器（Adam vs SGD） 6

5.6 （五）exp2 → exp5：学习率 6

5.7 （六）exp2 → exp6 / exp7 / exp8 / exp9：网络深度 7

6 可解释性分析（Grad-CAM） **8**

6.1 方法说明 8

6.2 分析与讨论 8

7 结论 **9**

1 实验目标

基于 baseline CNN，在 STL10 数据集上开展优化实验，尝试在模型训练过程当中可能会影响模型效果的因素，更好地掌握上课学习到的内容重点比较不同因素对模型效果的影响，包括：

- 数据增强是否提升泛化能力；
- BatchNorm 与 Dropout 对收敛和过拟合的影响；
- 优化器与学习率设置对最终测试表现的影响；
- 更深网络（`improved_long`，多段池化）相对浅层 configurable 的收益。

同时还通过 Grad-CAM 进行可解释性的分析，更好地理解模型为什么能够做出这样的预测

2 实验设置

- 数据来源：`STL10/train` 与 `STL10/test`。（课程提供）
- 训练配置：`batch_size=128`、`val_ratio=0.1`、`seed=42`。训练集按固定随机种子划分出验证集，验证集用于选择最佳 checkpoint；如果验证集准确率连续 `patience=20` 个 epoch 没有刷新最优值，则提前停止训练。
- 评估指标：Accuracy、Macro-F1、Weighted-F1、分类报告、混淆矩阵。
- 输出目录：`outputs_improved/`。（可在压缩包或 github 上查看）

3 实验分组与网络结构说明

实验在 baseline 上共 9 组改进设置，覆盖数据增强、BatchNorm、Dropout、优化器、学习率、网络深度，以及在同配方下增加卷积层数等。

- baseline：基础 CNN，无增强、无 BN、无 Dropout（Adam, lr=0.001, L2 `weight_decay=5e-4`）
- exp1_aug：数据增强（RandomCrop + HorizontalFlip + ColorJitter）
- exp2_aug_bn：数据增强 + BatchNorm
- exp3_aug_dropout：数据增强 + Dropout(0.3)
- exp4_aug_bn_sgd：与 exp2 相同（增强 + BN、无 Dropout），优化器改为 SGD（`lr=0.01`）
- exp5_aug_bn_lr：与 exp2 相同，Adam 学习率改为 `3e-4`
- exp6_improved_long_6：与 exp2 相同，卷积层数改为 6
- exp7_improved_long_9：与 exp2 相同，卷积层数改为 9
- exp8_improved_longer_12：与 exp2 相同，卷积层数改为 12
- exp9_aug_bn_15conv：与 exp2 相同，卷积层数改为 15

4 实验结果概览

下面汇总各实验组结果。baseline 测试准确率为 55.90%，`exp6` 最高，为 81.80%。详细的训练分析在后面会给出

实验组	Best Val Acc(%)	Test Acc(%)	Macro-F1	Weighted-F1
baseline	61.29	55.90	0.5594	0.5594
exp1_aug	74.00	68.30	0.6858	0.6858
exp2_aug_bn	77.14	74.60	0.7461	0.7461
exp3_aug_dropout	74.71	72.20	0.7239	0.7239
exp4_aug_bn_sgd	76.71	71.10	0.7115	0.7115
exp5_aug_bn_lr	76.43	73.30	0.7316	0.7316
exp6_improved_long_6	86.00	**81.80**	**0.8196**	**0.8196**
exp7_improved_long_9	83.71	81.40	0.8150	0.8150
exp8_improved_longer_12	84.57	81.20	0.8120	0.8120
exp9_aug_bn_15conv	80.86	77.30	0.7744	0.7744

表 1: 各实验组性能对比（见 `comparison_summary.csv`）

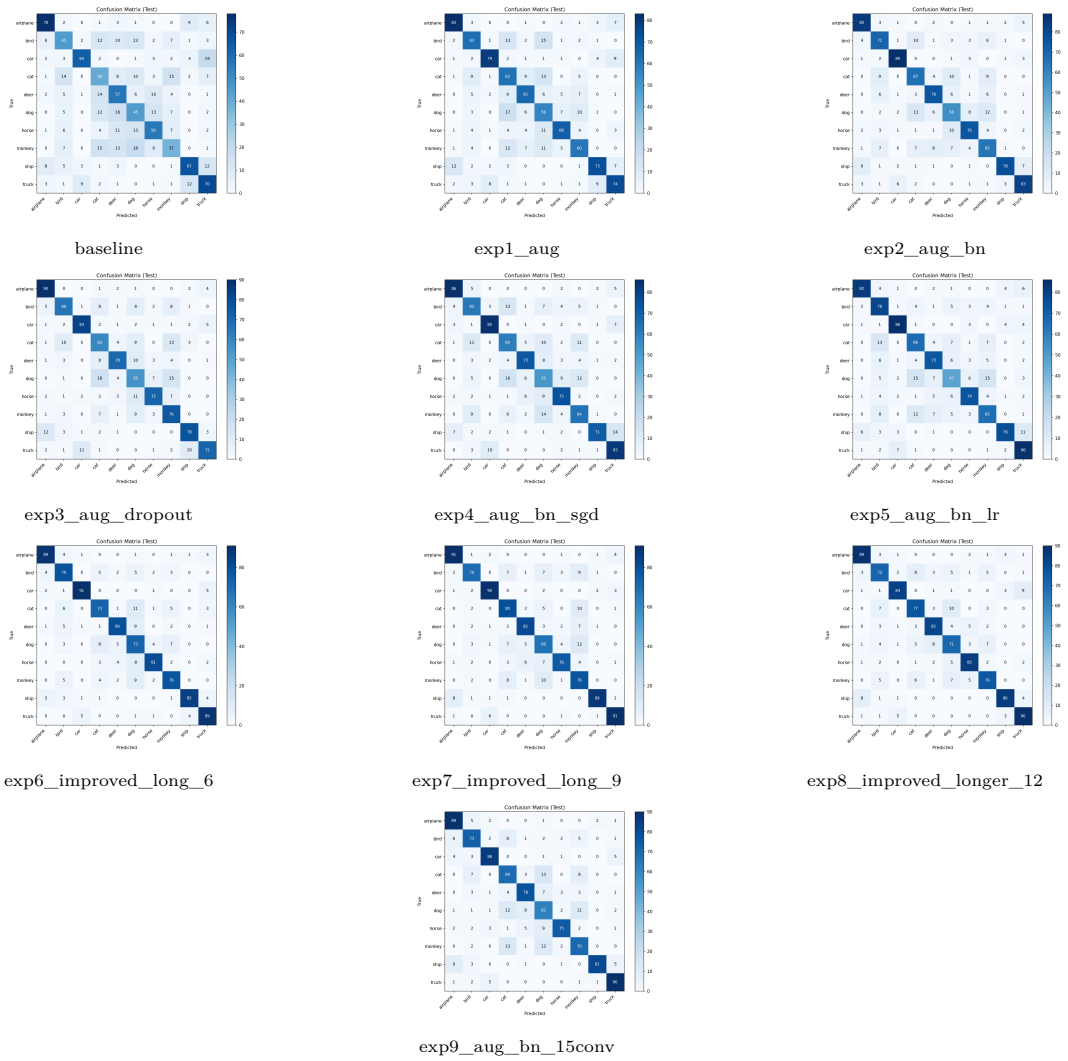


图 1: 测试集混淆矩阵（上三行 3×3 为 baseline 与 exp1–exp8；末行居中为 exp9）

5 训练过程与结果分析

本节先单独给出 **baseline** 在最终测试集上的多分类指标（Precision、Recall、F1 与混淆矩阵），再按对照分组给出训练曲线：每张图从左到右是训练/验证的 loss 和准确率，虚线是参照组、实线是对照组。汇总数字以表 1 为准，曲线主要看收敛快慢以及训练集与验证集差距。

需要注意的是，本实验多数设置只在固定 seed 下运行一次，因此结果仍可能受到随机初始化和数据划分的影响。不同模型之间 0.2%–0.5% 的差距不一定代表稳定优势。

5.1 Baseline：测试集分类报告与混淆矩阵

Baseline 为无数据增强、无 BatchNorm、无 Dropout 的浅层 CNN（Adam，`lr=1e-3`）。在 STL-10 官方测试集（每类 100 张、共 1000 张）上，整体测试准确率为 **55.90%**，与验证集相比仍有明显差距，说明存在过拟合。表 2 给出各类别的 Precision、Recall 与 F1-score；图 2 为对应的测试集混淆矩阵

类别	Precision	Recall	F1	Support
airplane	0.7723	0.7800	0.7761	100
bird	0.4839	0.4500	0.4663	100
car	0.8101	0.6400	0.7151	100
cat	0.3883	0.4000	0.3941	100
deer	0.4672	0.5700	0.5135	100
dog	0.4245	0.4500	0.4369	100
horse	0.5773	0.5600	0.5685	100
monkey	0.4568	0.3700	0.4088	100
ship	0.7053	0.6700	0.6872	100
truck	0.5691	0.7000	0.6278	100
macro avg	0.5655	0.5590	0.5594	1000
weighted avg	0.5655	0.5590	0.5594	1000

表 2: Baseline 在 STL-10 测试集上的逐类 Precision、Recall、F1 及宏平均（Accuracy = 0.5590）

从表 2 可以看出，baseline 对 `car`、`ship`、`airplane` 这类外形比较明显的类别判断得相对好一些，Precision 或 Recall 比较高；但对 `cat`、`dog`、`monkey` 这类更相似的动物类别，效果就差一些。说明这个浅层 CNN 能学到一些整体轮廓特征，但还不太擅长区分细节，比如纹理、局部部件这些更细的差别。

由于 STL-10 测试集中每个类别都是 100 张，类别比较均衡，所以 Macro-F1 和 Weighted-F1 数值接近，也和 Accuracy 比较接近。再结合图 2 可以看到，混淆矩阵里非对角线位置还有不少数值，说明模型经常把一些类别互相认错。这也说明只用 baseline 还不够，后面继续尝试数据增强、BatchNorm 和更深网络是有必要的。

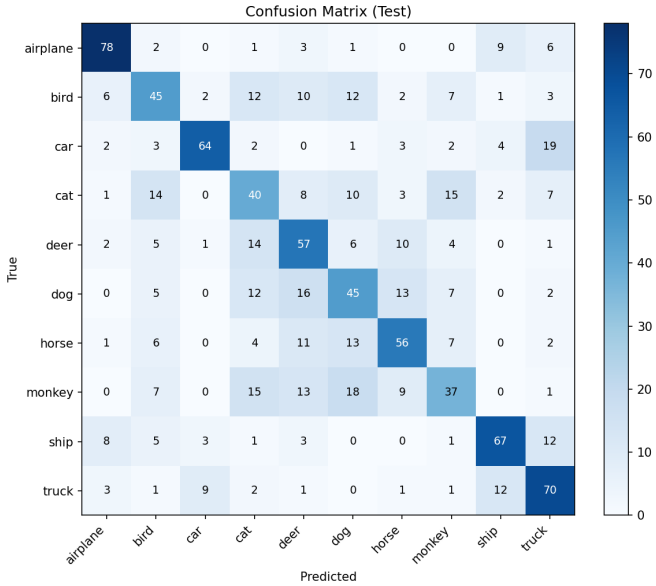


图 2: Baseline 在 STL-10 测试集上的混淆矩阵

5.2 （一）baseline → exp1：数据增强

- baseline 测试为 55.90%，exp1 为 68.30%，成功率提升明显
- 从曲线上看，baseline 的过拟合非常明显，exp1 的过拟合程度明显减轻，说明数据增强确实有效果

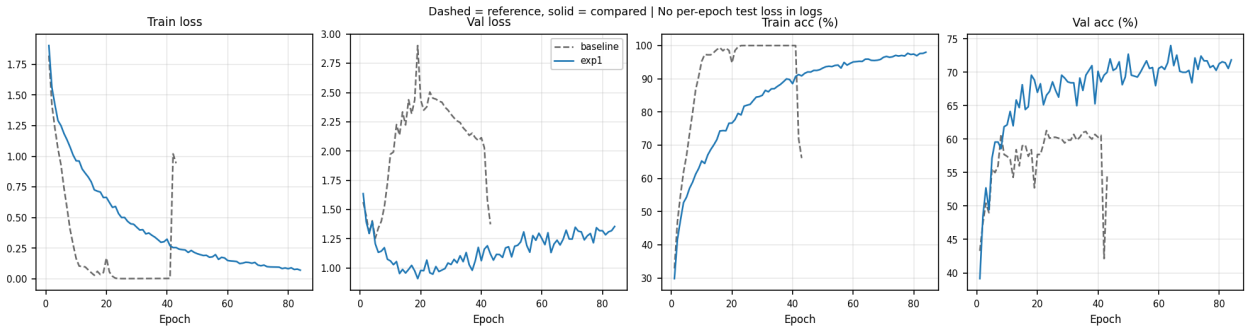


图 3: baseline（虚线）与 exp1_aug（实线）：训练/验证 loss 与准确率

5.3 （二）exp1 → exp2：BatchNorm

- exp1 测试 68.30%，exp2 为 74.60%，在已有增强的前提下加入 BatchNorm 后模型的效果明显提升，并且后面很多实验都在 exp2 的基础上进行。

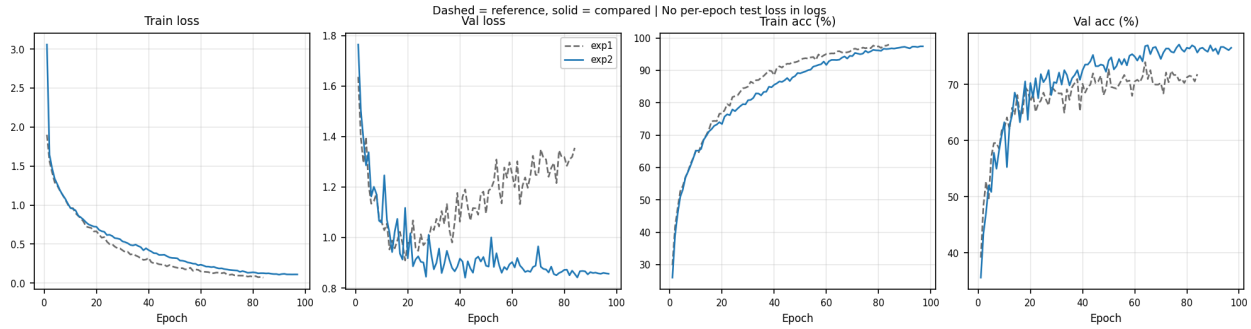


图 4: exp1_aug（虚线）与 exp2_aug_bn（实线）

5.4 （三）exp2 → exp3：Dropout（与 BN 路径对照）

- exp2 测试 74.60%，exp3 为 72.20%，略低于 exp2，但均远高于 baseline，说明两条正则化路径都有效，差距不大。

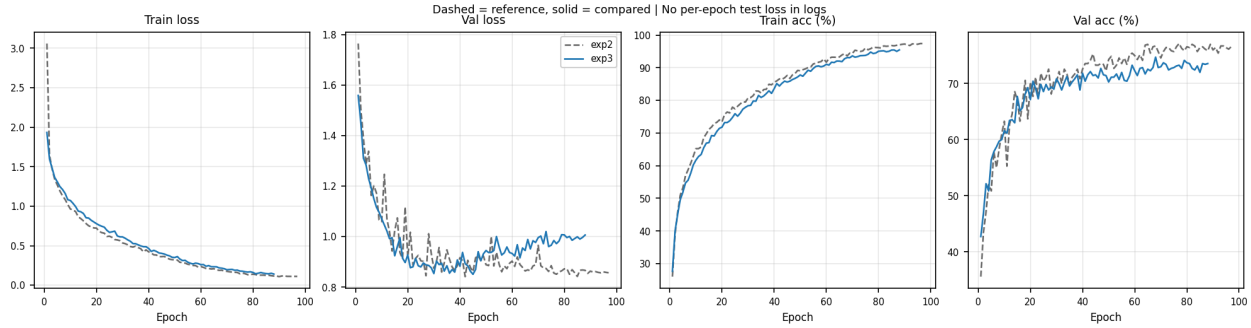


图 5: exp2_aug_bn（虚线）与 exp3_aug_dropout（实线）

5.5 （四）exp2 → exp4：优化器（Adam vs SGD）

- 不同优化器对比：exp2（Adam）测试 74.60%，exp4（SGD）71.10%，Adam 略优。

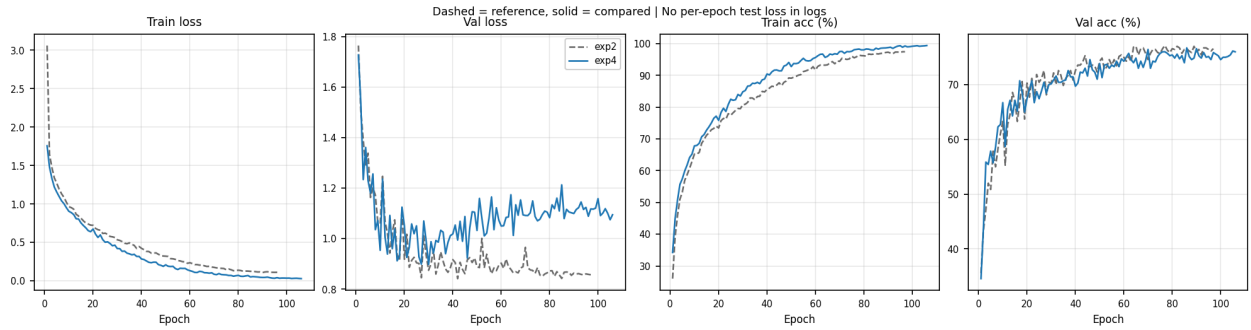


图 6: exp2_aug_bn（虚线）与 exp4_aug_bn_sgd（实线）

5.6 （五）exp2 → exp5：学习率

- exp2 用 `1e-3`，exp5 改成 `3e-4`，测试从 74.60% 降到 73.30%，说明并不是学习率越小越好。

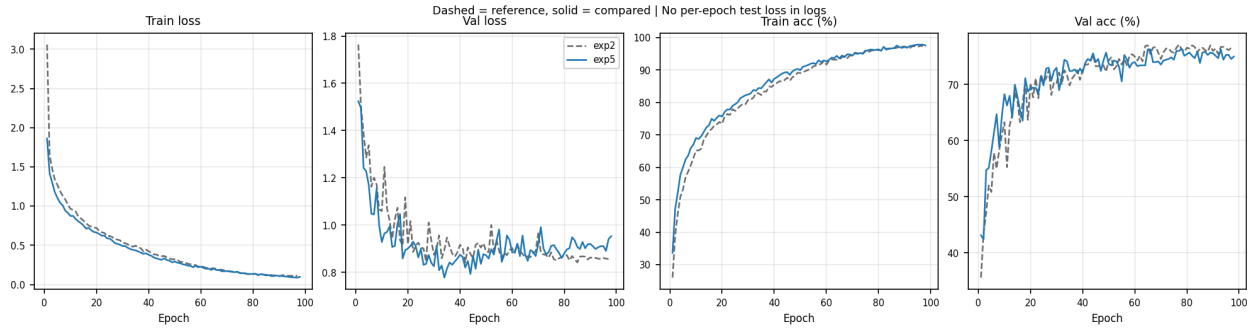


图 7: exp2_aug_bn（虚线）与 exp5_aug_bn_lr（实线）

5.7 （六）exp2 → exp6 / exp7 / exp8 / exp9：网络深度

- 浅层 exp2 测试 74.60%；加深后 exp6 达 81.80%、exp7 为 81.40%、exp8 为 81.20%，说明多段池化的深层骨干在本数据上明显优于浅层 configurable。
- exp8 相对 exp6/exp7 未再带来额外增益，但仍保持与 exp7 接近的测试水平，说明继续加深并不保证单调提升，但也未必立刻崩溃。
- 另设 exp9（与 exp2 同配方、15 个卷积层），测试 77.30%，明显低于多段池化骨干 exp6（81.80%）；下图将二者曲线对齐后可见，exp9 的验证准确率整体低于 exp6，**测试准确率也下降约 4.5 个百分点**，说明并非「层数堆得越多越好」。

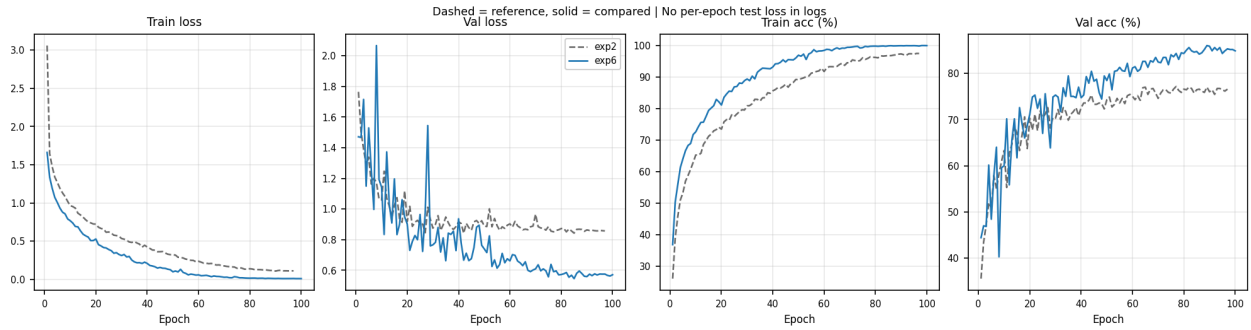


图 8: exp2_aug_bn（虚线）与 exp6_improved_long_6（实线）

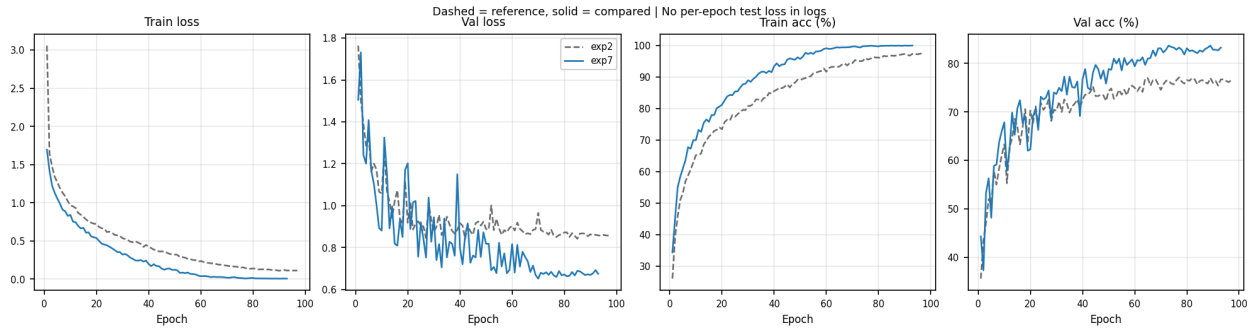


图 9: exp2_aug_bn（虚线）与 exp7_improved_long_9（实线）

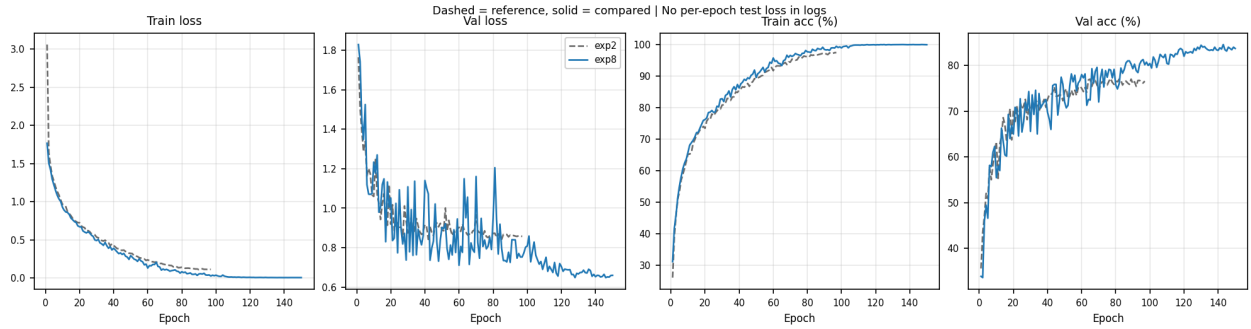


图 10: exp2_aug_bn（虚线）与 exp8_improved_longer_12（实线）

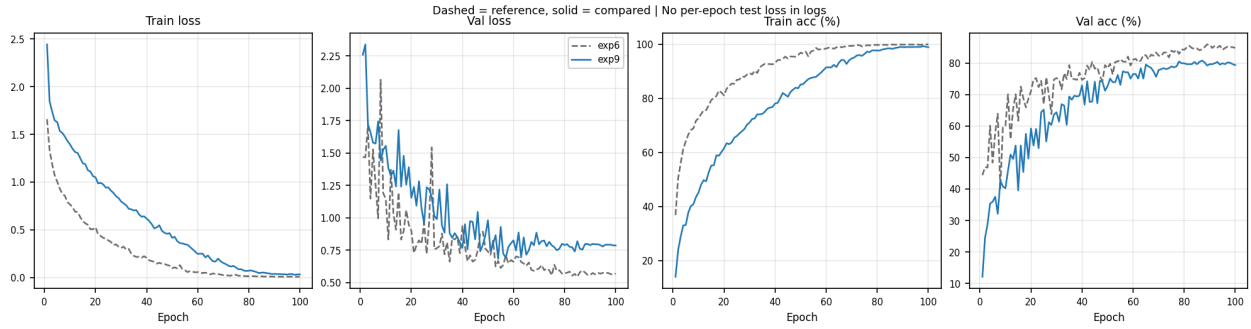


图 11: exp6_improved_long_6（虚线）与 exp9_aug_bn_15conv（实线）：exp9 验证与测试表现均低于 exp6，测试准确率由 81.80% 降至 77.30%

6 可解释性分析（Grad-CAM）

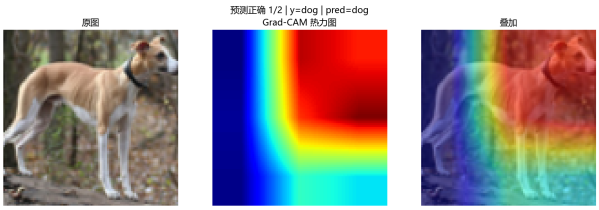
6.1 方法说明

Grad-CAM 的作用是看模型做判断时主要关注了图片里的哪些位置。它会利用最后一层卷积特征图和类别得分的梯度，算出不同通道的重要程度，再合成一张热力图。热力图越亮的地方，说明模型在预测这个类别时越依赖这些区域。

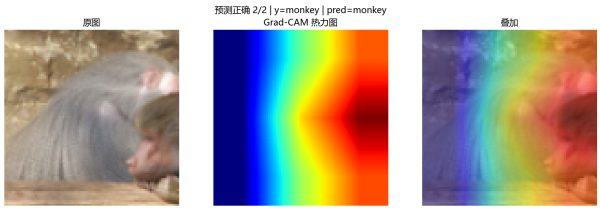
本实验没有改动网络结构，只是在最后一层 `Conv2d` 上取特征和梯度。这里解释的是模型当前预测出来的类别，也就是 $\arg\max$ 对应的类别。输入图片的预处理方式和测试阶段保持一致，包括 Resize、CenterCrop 和 Normalize(0.5)。

6.2 分析与讨论

模型选用当前测试集表现最好的 `exp6_improved_long_6` 对应的 checkpoint。样本来自 STL-10 官方测试集；为便于对比，从测试集中分别选取 **2** 个预测正确与 **4** 个预测错误的样本做 Grad-CAM。每张导出图均为「原图—热力图—叠加」三列排版。下文选取 2 张正确样本和 4 张错误样本作为主图展示；脚本也会把更多结果分别保存到 correct 与 wrong 子目录，便于复查。



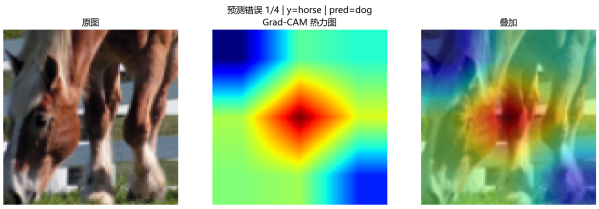
(a) 预测正确样本 1



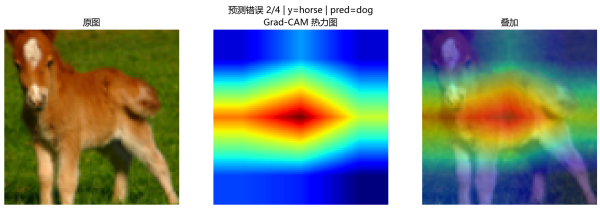
(b) 预测正确样本 2

图 12: 预测正确时的 Grad-CAM：热力图多集中在物体主体与轮廓、纹理等可区分区域。

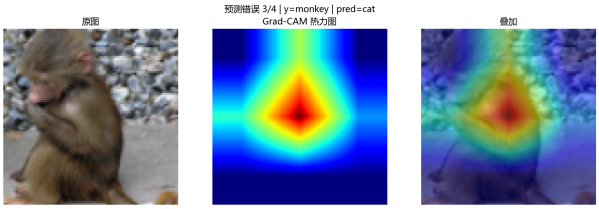
从上述两个正确样本可以看出，模型比较集中地看到了物体主体，比如轮廓、纹理这些比较有辨识度的地方。这说明模型确实学到了一些和类别有关的图像信息，而不只是记住背景颜色或者图片位置。



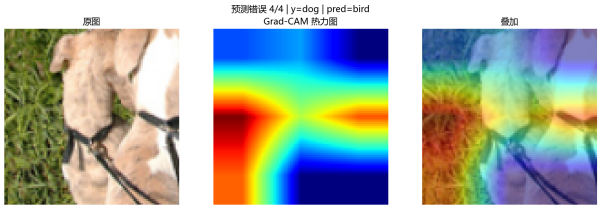
(a) 预测错误样本 1



(b) 预测错误样本 2



(c) 预测错误样本 3



(d) 预测错误样本 4

图 13: 预测错误时的 Grad-CAM：注意力更易分散，或受背景、边缘干扰，或对关键部位响应不足，从而与相似类别混淆。

错误样本的热力图就没有这么理想。多个错误例子里，有时模型关注的位置比较散，或者被背景、边缘这些地方吸引；有时它也看到了物体附近，但抓到的特征不够准确，所以会把相似类别混在一起。也就是说，模型已经能找到一些有用区域，但还没有稳定到每次都能抓住真正关键的部分。

7 结论

本次课程项目完成了 STL10 图像分类任务中的 baseline 与多组改进实验。整体结果来看，数据增强和 BatchNorm 对浅层模型提升比较明显；继续加深网络后，`exp6_improved_long_6` 取得了本实验中最高的测试准确率 **81.80%**。同时，`exp9_aug_bn_15conv` 的结果也让我看到，单纯增加卷积层数不一定就会更好，模型结构、池化方式、训练轮数和正则化都需要一起考虑。

这次项目对我来说最大的收获不是只跑出了一个最高准确率，而是比较完整地走了一遍深度学习实验流程。五一期间我集中学习了 PyTorch，从一开始看不太懂 `Dataset`、`DataLoader`、`forward` 和训练循环，到后面能够自己修改模型结构、调整实验配置、重新训练、整理曲线和分析结果，这个过程让我对卷积神经网络的训练有了更具体的认识。Grad-CAM 的可视化也让我意识到，模型的准确率只是一个方面，还需要看它到底关注了图像里的哪些区域，才能更好地理解模型为什么会预测正确或出错。

最后，非常感谢老师在课堂上对人工智能原理和神经网络方法的讲解，也感谢助教李自远学长在课程项目中的答疑和指导。这个项目让我把课上学到的概念真正放到代码和实验里跑了一遍，也让我对后

续继续学习人工智能原理更有信心。

附件与提交说明

按课程要求整理代码、说明文档与实验报告。项目编程语言为 Python，主要依赖 PyTorch 与 torchvision。训练、测试、结果汇总和可解释性分析代码均放在 `src/` 目录下。本次作业已上传 github 仓库，地址为：`https://github.com/hezhou0331/CNN_learning.git`

本次主要使用 OpenAI Codex 辅助完成模型搭建、实验脚本调整、批量运行实验、结果汇总、曲线绘制等工作。实验设计思路、最终结果选择、报告中的分析判断和结论由本人结合课程要求与实际实验结果完成。

模型结构主要参考课堂中介绍的 CNN 基本思想，以及 PyTorch 官方模块中 Conv、Pooling、BatchNorm、Dropout 等常见层的用法自行搭建。